

Author: Andrea Mussari

Part 1:

In this part the fractions includes the floor operation on the result of division.

1)

If W is a prefix of S the computation of F(W) with only F(S) and |W| is impossible because:

$$F(W) = \frac{F(S)}{131^{|S|-|W|}}$$

2)

$$F(S) = F(W) * 131^{|W|} + F(Z)$$

$$F(W) = \frac{F(S) - F(Z)}{131^{|Z|}}$$

$$F(Z) = F(S) - F(W)$$

Part 2:

1. Preprocessing Phase:

- Compute Karp-Rabin fingerprints for all strings $S_1, S_2, \dots, S_m$ in dictionary D
- Store these fingerprints in a hash table for O(1) lookup
- Precompute powers of the base number used in the hash function

2. Main Data Structure Components:

- Hash table of dictionary string fingerprints
- Rolling hash representation of the current text T
- Window-based scanning system to check for matches

3. Dynamic Text Management:

- Maintain a rolling hash for all K-length substrings of T
- After each edit operation, efficiently update only the affected hashes

For each edit operation, we need to update the rolling hashes of all affected K-length windows:

1. Insert operation: When a character is inserted at position i in T:

- Update the hash values for all windows that include position i
- This affects K windows starting from position i-K+1 to i

2. **Delete operation:** When a character is deleted at position i in T :

- Update the hash values for all windows that included position i
- This affects K windows starting from position $i-K+1$ to i

3. **Replace operation:** When a character at position i is replaced:

- Update the hash values for all windows that include position i
- This affects K windows starting from position $i-K+1$ to i

Space Complexity:

- Dictionary fingerprints: $O(M)$ space for M strings
- Rolling hash windows: $O(|T|)$ space for text of length $|T|$
- Additional precomputed values: $O(K)$ space
- Total space: $O(M + |T| + K)$

Time Complexity:

- Preprocessing: $O(M \cdot K)$ to compute all dictionary string fingerprints
- After each edit operation:
- Hash updates: $O(K)$ time to update affected rolling hashes
- Matching: $O(1)$ lookups in the hash table per window, with $O(K)$ windows to check
- Total per edit: $O(K)$

Optimization Considerations:

- The data structure can be further optimized to handle very large dictionaries by using bloom filters as a first-pass filter

The main advantage of this approach is that after an edit operation, we only need $O(K)$ time to update the affected hashes and check for matches, instead of rescanning the entire text, which would take $O(|T| \cdot M)$ time using naive methods.

Part 3:

$$S = \{a, b, c\}$$

$$h_1(a) = 0, h_2(a) = 1$$

$$h_1(b) = 0, h_2(b) = 2$$

$$h_1(c) = 1, h_2(c) = 0$$

Example:

1. insert a : [a,-,-],[-,-,-]
2. insert b : [b,-,-],[-,a,-]
3. insert c : [b,c,-],[-,a,-]
4. deletion b : [-,c,-],[-,a,-]
5. Insert S - {b} = {a,c}
 1. insert a : [a,-,-],[-,-,-]
 2. insert c : [a,c,-],[-,-,-]

After the deletion of b the position of a is changed.

By randomly choosing between 1 and 2 for the first attempt to insert an element we break the deterministic pattern and therefore make the reinsertion less dependent on previous insertions.